SECURE
SOFTWARE SYSTEM
(MITS 5400G/CSCI 6320G)

Submitted to,

Name: Dr. Pooria Madani

Faculty of Business and Information Tech

Submitted by,

Name: Avishkar Sharma

Assignment 1

Contents

Q.NO.1

Comparison and Contrast:

Features	Waterfall Model	Scrum Model
Approach	It is sequential, phase-wise developmental approach	It is based on iterative and incremental development.
Requirements	Stable and are defined at the very start of the project	Evolving and flexibility where tasks are prioritized according to backlog
Security Identification	Either early or even very late	Automatically integrated in every sprint
Risk	High risk of late discovery	Lower risk due to early and continuous testing
Objective	Documentation and initial plan	Providing the priorities features first to the user

Table 1: Comparison between waterfall and scrum model

Conclusion

Waterfall goes well with projects having definite needs and requirements for tight documentation; it reveals security issues late because security is bolted on. While Scrum on the other hand is a technique where the requirements are changing and security are integrated based on backlog throughout the phases which helps to identify and minimize the vulnerabilities from the early stage. Waterfall model is a linear process. Scrum on the other hand, is an iterative and incremental approach aimed at creating resilience and integrating it continuously with frequent feedback and transformation.

Q.NO.2

Test Driven Development (TDD):

Test-Driven Development is quite interesting approach where the developer writes the test cases first, even before actual coding and starts failing them, and later write minimal code then pass them.

Spiral Model:

In this model, testing is done after the development in each cycle of the Spiral Model. Its major concern is with functionality confirmation and reduction of risk, particularly in large and critical systems.

Comparison between Test Driven Development and Spiral Model Testing

Aspect	Test Driven Development	Spiral Model Testing
Error Identification	Before implementation	After development phase
Risk Management	Focuses mainly on code correctness	Risk analysis is the core feature
Documentation	Minimal	Heavy for risk analysis and planning
Development style	Agile and incremental	Iterative and risk-driven
Purpose	Preventing defects early	Detect and control project risks

Table 2: Comparison between Test Driven and Spiral Model

Conclusion

TDD is a proactive development technique where testing enables the developer to make coding decisions, where Spiral process represents structured validation that manages risks for complex and large-scale projects. A clear realization of these differences will help software engineers to define the proper development and testing strategy for a project.

Q.NO.3

Analysis of the Provided Code:

The code provided performs an access control check by evaluating the return value of security-relevant function. It does this by first making the function call, which calls the access control decision function *ISAccessAllowed(...),* and the value is returned to *dwRet*. The determination logic will deny access only if this function explicitly *returns if (dwRet == Error_Access_Denied)*; otherwise, it assumes the security check succeeded and is allowed to go through. This means the code treats unexpected return values, or even errors, as successful authorizations.

Fail Safe Defaults Principal Violations

The behaviors above are just in opposite line of the principle of fail-safe defaults as proposed by Saltzer and Schroeder. They say that, in a system, access control decisions should be very conservative and explicitly deny access until access has been explicitly granted.

Security Implications

Permitting access on error or undefined conditions allows unauthorized access, decreasing the security mechanism's effectiveness. Such logic leads to an enlarged attack surface and might be exploited by the attacker.

What I did in the program:

```
DWORD dwRet = IsAccessAllowed(...);

if (dwRet == ERROR_SUCCESS) {
    // Security check is OK - grant access
} else {
    // Security check failed - deny access
}
```

Figure 3.1: Changes in the code provided

By implementing the above function, one's default access is denied and allows it on positive confirmation only; a call to the function *if (dwRet == Error_success)* virtually implements the Fail-Safe default principle. This would ensure the system is kept secure even in the case of errors or unexpected conditions.

Q.NO.4

The proposed IoT Blood Pressure Monitoring System is intended for a hospital setup and continuously monitors inpatients through IoT blood pressure sensors. In this type of safety-critical environment where system availability remains as one of the notable key requirements. All possible threats to availability that have been recognized by the Microsoft Threat Modeling Tool fall under the category of Denial of Service (DoS).

Threat 1:

ID: 17 (Blood pressure measurements transferred to the Web Application.)

Description: Any submission of blood pressure measurements by Client may result in a potential crash or slowdown of the Web Application due to malformed inputs, excessive traffic, or unhandled exceptions. Staff cannot submit or view patient readings, delaying monitoring and potentially affecting patient safety.

Mitigation

1. Input validation and exception handling can be implemented, which will ensure that bad or malformed data does not crash into the Web Application.

2. Chaos Engineering tests like server crash simulation or high traffic spike test should be done to make sure the system behaves predictably under control failures.

3.AI-based Web Application Firewall to monitor traffic and automatically block abnormal or potentially malicious requests this will deny a lot of unwanted or malformed requests.

Threat 2

ID: 18 (Web Service Crashes, Fails, Stops Working or slow)

Description: In this process, the blood pressure measurements must be sent from the Client to the Web Application. An external agent could interrupt or block the data flow across the network. If the data doesn't reach to the Web Application, then readings of patients will simply be delayed or even missed, hence decreasing system availability.

Mitigations:

1. TLS/HTTPS encryption should be enabled to protect the data in transit and ensure tampers are minimal, making any intrusion more difficult.

2. HTTP client side retries mechanisms with exponential backoff using tenacity, will ensure temporary failed requests do not hamper the disruption of data submission.

3.Implementation of network monitoring tools like Grafana or Nagios detect sudden drops or breaks in traffic.

Threat 3:

ID: 56 (Potential Process Crash or Stop for Web Service)

Description: In this process, readings of blood pressure are transferred from the sensor to the Web Service through the RPC connection. The service may begin crashing or stopping, with slow operation being experienced because of malformed payloads from the sensor, uncontrolled concurrency or due to internal faults of the service. This translates to a delayed or totally unavailable data feed for monitoring patients.

Mitigation

1. Schema validation using Protocol Buffer or any similar tools can ensure that only rightly structured sensor data is ingested, thus avoiding crashes due to malformed RPC payloads.

2. Bulkheading service threads can isolate sensor processing workloads so that any failures in one thread don't impact on the Web Service as a whole.

Threat 4:

ID: 57 (RPC sends pressure readings to Web Service from Sensor)

Description: This may cause downtime in communication between the Sensor and the Web Service. Real-time readings of blood pressure may not achieve back-end services; this will, therefore, result in low availability of systems with a consequence of deferral of clinical decisions.

Mitigation

1. Edge buffering with intelligent sync will store readings and pushes once connected; this can be done with either local storage or flash memory.

2. Software-defined networking traffic can prioritize RPC packets over other types of data and define QoS rules that medical data must reach their destination first.

Threat 5:

ID: 67 (RPC sends Sensor pressure read to Web Service)

Description: The process where the Web Application user input saves into an SQL Database. If over-submission and badly optimized queries are executed it will burden the database or web application, it would slow down or even be partially unavailable, thus delaying operations for the staff.

Mitigation

1. Query optimization and indexing can be implemented as it adds speed to database operations, preventing slowdowns.

2. Frequent queries can be served from memory through caching, avoiding database load so we can implement Redis server or even load balancing can be an option.

3. Autoscaling and resource orchestration is another option as it dynamically allocates CPU and memory resources to handle spikes in submissions.

Threat 6:

ID: 70 – Potential Excessive Resource Consumption for Web Service or SQL Database.

Description: Patient's blood pressure readings taken through Web Service are saved back to a SQL Database.

Impact: In the case of high-volume automatic readings, the database or the Web Service might get flooded, which may slow it down or become temporarily unavailable, thereby delaying access to critical data about a patient.

Mitigation

1. Time series optimized storage structure can be used to optimize the schemas or engines such that it improves writing efficiency and reduces resource saturation.

2. Real-time monitoring can also be solution as it uses an adaptive method for throttling so that system resources don't get overwhelmed.

Q.NO.5

Q.NO.5(a) Attack nodes and Probability of success

Attack Node	Probability	Justification	Source
Phishing Email	0.6	In the education sector, phishing mail is the most common method that serves as entryway for the attacker. Based on drib summary its high prevalence in breaches reports, phishing is assigned a higher probability success.	https://thebestvpn.com/statistics
Credential Reuse	0.03	Most of the students have the habit of reusing the passwords for multiple servers which can then lead to third party breaches	https://www.wiz.io/academy/
Email Spoofing	0.21	Spoofed emails can easily deceive anyone to get into the trap this result in moderate likelihood of success.	https://aag-it.com/the-latest-phishing-statistics/
Social Engineering	0.35	One of the common methods is to impersonate IT staff or Faculty win victim's trust and obtain credential by making this attack highly effective.	https://secureframe.com/blog/social-engineering-statistics
Malicious Link	0.30	Despite Awareness Training, users still click on mail where the malicious links could be embedded.	https://www.jerichosecurity.com/blog/
Brute Force Attack	0.17	Rate limiting and account lockout significantly reduces possibility of this technique resulting in a lower probability of success.	Personal Judgement
Credential Stuffing	0.21	This attack depends on previous credential leakage; their success highly depends on password reuse which remains common.	https://specopssoft.com
MFA Fatigue Attack	0.09	This has a very simple requirement where one must reveal a credential which is uncommon thus decreasing the overall success rate of the attack.	https://techcommunity.microsoft.com/blog/microsoft-entra-blog/

Unattended Device Access	0.25	Use of shared and public places like libraries and labs can increase the danger of unattended authenticated devices making it a possible success.	Personal Judgement
Access Before Session Timeout	0.20	There is always the potential that an unattended authenticated device can allow access to email if one has left a session active and it is yet to encounter the automatic timeout mechanism	Personal Judgement
Victim Logged In	0.05	Users generally have long session time, which can create high possibility of a session-based attack, but achieving both case is very rare	Personal Judgement
Session Token Captured	0.80	Though, session token is encrypted and can't be stolen but there can be malware implanted once the attacker gets the session there is no way of protection, but whole process of achieving is very difficult	https://blackhat.com/presentations/bh-usa-04/bh-us-04-shema-up.pdf

Table 5.1: Attack nodes and Probability of success

Exact value for all the attacks is not explicitly in the referenced sources, so some of the values are derived from the reading of attack prevalence, effectiveness and personal judgement and experience in the cybersecurity field.

Q.NO.5(b) Defense Nodes with Effectiveness

Defense Node	Effectiveness Probability	Justification	Source
Phishing Awareness Training	0.91	Awareness training and simulation heavily decreases the vulnerability to phishing attack	https://futurecisco.tech/security-training
Multi-Factor Authentication (MFA)	0.99	MFA mostly reduces the risk even when the password is compromised as attackers have to bypass another layer	https://cloudmails-my.sharepoint.com/
DMARC	0.96	This mechanism will prevent domain spoofing and reduce the success of phishing significantly	https://powerdmarc.com/email-phishing-dmarc-statistics
AI Driven Behavioral Analysis	0.86	It can detect anomalous login behavior like location or access time so early detection can be implemented which increases effectiveness	Personal Judgment
URL Sandboxing	0.90	It monitors and analyzes the suspicious link thus preventing from phishing or malware websites	https://www.sciencedirect.com/science/
Account Lockout	0.82	Limiting the number of failed logins by user or attacker can prevent frequent automated attack	Personal Judgement
Breached Password Audit	0.67	This reduces the success of credential stuffing as it prevents us from reusing previously compromised passwords from multiple sources.	Personal Judgment
MFA Rate Limiting	0.99	MFA is highly effective, it can stop attacks including brute force and password spraying, but for MFA fatigue additional defenses are required.	https://cdn-dynmedia-1.microsoft.com/

Conditional Access/ IP reputation	0.99	It restricts login attempts based on the location, devices trust or IP so even with valid credential one can't access and the best way is to be merged with MFA.	https://www.pax8.com/blog/conditional-access/
Presence Detection	0.74	The screen will auto lock if the owner of device moves away which will protect unattended devices, this uses hardware like IR sensor or web cams which may not be supported by old devices	Personal Judgement
Session Timeout	0.80	They can prevent and mitigate risks like unlimited access sessions and make session hijacking less successful.	Personal Judgement
Endpoint Detection and Response	0.95	This solution monitor endpoints and detects suspicious behavior or even malware execution making it a very effective approach	https://www.vectra.ai/topics/endpoint-detection-and-response
TLS Enforcement	0.95	TLS implementations make intruder almost impossible, however it's not best against the attack like malicious payload hidden in encrypted session.	Personal Judgement

Table 5.2: Defense Nodes with Effectiveness

All the effectiveness probabilities are not publicly available for all defensive node, so values are derived and computed from personal perspective also followed some documents for effectiveness tends.

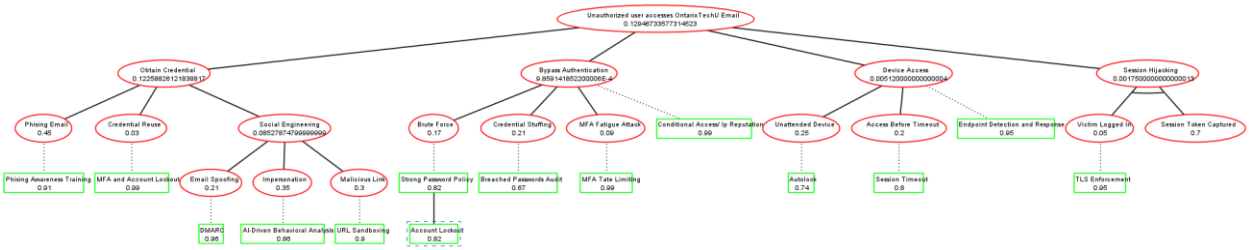

Figure 5.1: AD Tree from the point of view of Defender to Opponent

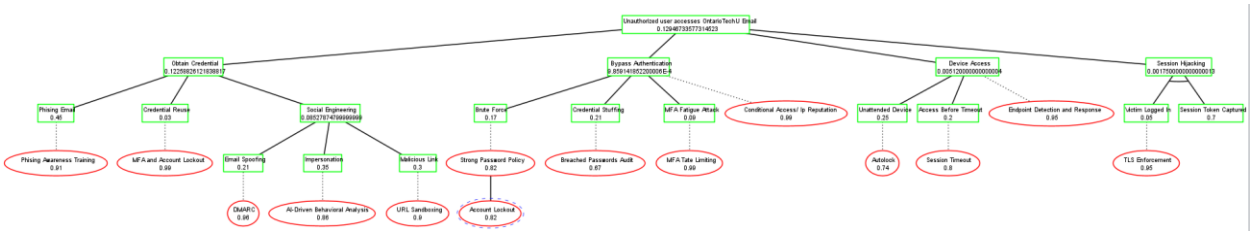

Figure 5.2: AD Tree from the point of view of Defender to Proponent

Above Attack-Defense tree is a dynamic view of security landscape from the point of view of attacker and defender. No security control is impenetrable, and no attack vector is guaranteed to succeed. At the top we have the root (Unauthorized user access to Ontario Tech Email) with the probability of 0.129, which means there is chance of 12.9% for unauthorized access, usually at a very low level. The most important threat vector here is the Obtain credential branch with 0.122, showing that humans are the weakest link, in this case social engineering and phishing are the main vulnerabilities. The tree mitigates such risk with advanced defense mechanisms, that include Awareness, MFA, AI-Driven behavioral analysis and URL sandboxing to minimize the success rate. In the Appendix section, all nodes type, label and assigned probability are attached.

Q.NO.6

Q.NO.6(a) Buffer overflow Vulnerability Analysis:

Buffer overflow Vulnerability Analysis:

```
printf("Choose (R)ock,(P)aper, or (S)cissor: ");
scanf(" %s",&user_selection);
user_selection = toupper(user_selection);
```

Figure 6.1: scanf line

This Rock-Paper-Scissors program is vulnerable to a stack-based buffer overflow during execution of the *play ()*, resulting from the *scanf ()*. It reads in strings until arbitrary length, terminated by a *white space ' '* or *tab "/t"* or *newline "/n"*. Nevertheless, user selection has been declared as a single character variable.

```
char computer_selection;
char dummy_flag[2] = {'c','l'};
char user_selection;
char choices[3] = {'R','P','S'};
```

Figure 6.2: Variable declaration

This kind of misrepresentation basically lets more than one character of legitimate user input be longer than one character; it can go on overwriting subsequent memory locations on the stack, leading to stack corruption.

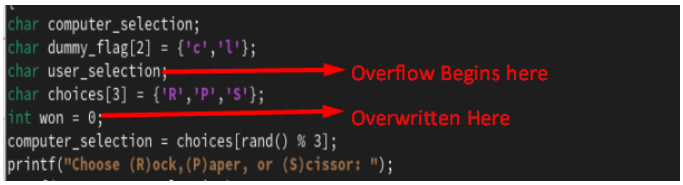

Figure 6.3: Stack and Variable Layout to be Affected:

Because stack memory is continuous, overflowing with *user_selection* will make it very easy for an attacker to overwrite many local variables, which include the integer variable *'won.'*

Before Buffer Overflow:

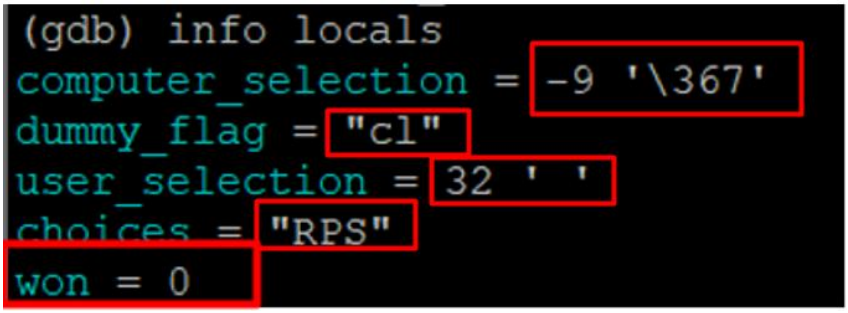

Figure 6.4: Stack variables before buffer overflow

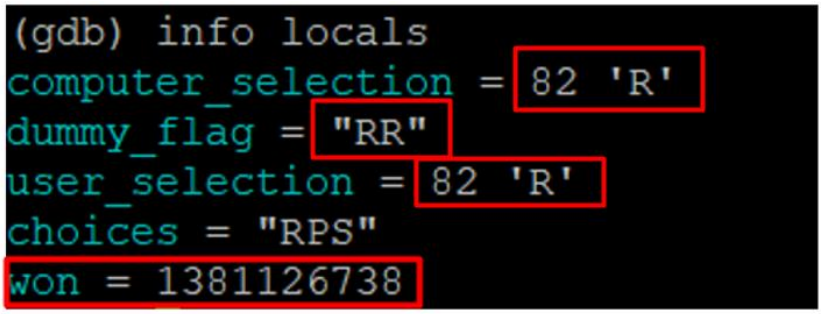

Figure 6.5: Stack variables after buffer overflow

After the Buffer Overflow, *dummy_flag='RR'* which shows overflow past allocated boundary, *computer_selection= 82 'R'* which is corrupted, same goes for *user_selection* as well. The *won* value changes to *non-zero* due to overwrite while *choices* are still intact as they were stored elsewhere in the memory. Above screenshots indicate that inserting overlong input overwrites the adjacent stack variable including the integer value of *won,* turning it into a non-zero value which forces the program to declare the winner always.

Q.NO.6(b) Exploitation of Buffer Overflow

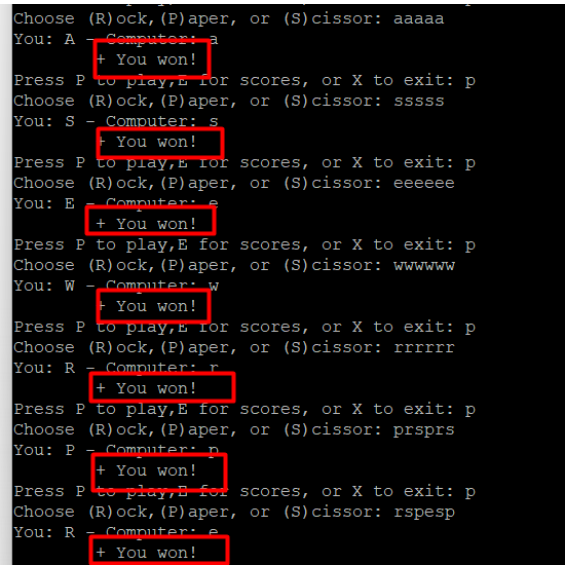

Figure 6.6: Successful exploitation

The above figure displays continuous executions of provided Rockpaperscissors program where overlong inputs like," rrrrr" or "aaaaa" or "sssss" or mixed strings are provided instead of a single character. Despite of different selection each time the program outputs "You Won", which demonstrate successful exploitation of the stack-based buffer overflow. This happens because of the overflow which overwrites the stack variable won with a non-zero value, that forces the win condition to evaluate as true every time.

Q.NO.6(c) Possible Solution for buffer OVerflow

The buffer overflow which was identified in Section 6.a is caused by unsafe input handling, where we had unbounded string input operation to a single byte variable, which was allowing user to overwrite adjacent stack memory. To fix this issue, the vulnerable input operation was replaced by type-safe option, and the program now ensures that user input can't corrupt stack variable, such that eliminating the buffer overflow exploitation factor.

```
printf("Choose (R)ock,(P)aper, or (S)cissor: ");
scanf(" %c", &user_selection);    // Read a single character safely, doesn't exploit buffer overflow
user_selection = toupper(user_selection);
```

Figure 6.7: Fix for buffer overflow vulnerability

While we could have taken alternative approach such as *bounded string input* or *fgets()* for the prevention of overflows, since this is a single-character input scenario so using *%c* is the most appropriate solution.

Conclusion:

Apart from the identified buffer overflow vulnerability, several other logical improvements were applied to enhance the overall correctness of the program, like input validation, handling tie conditions and other logical changes that can improve the overall experience of the game. The fully updated and corrected version of the program has been shared in the appendix section allowing the behavior of the revised codes to be reviewed and tested.

Appendix

Attack Defense Tree Node label and assigned value:

Node Type	Node Label	Assigned Value
Proponent	Impersonation	0.35
Proponent	Victim Logged In	0.05
Proponent	Unattended Device	0.25
Proponent	Phising Email	0.08
Proponent	Credential Reuse	0.03
Proponent	MFA Fatigue Attack	0.09
Proponent	Access Before Timeout	0.2
Proponent	Email Spoofing	0.21
Proponent	Malicious Link	0.3
Proponent	Session Token Captured	0.8
Proponent	Credential Stuffing	0.21
Proponent	Brute Force	0.17
Opponent	URL Sandboxing	0.94
Opponent	Account Lockout	0.7
Opponent	TLS Enforcement	0.95
Opponent	Phising Awareness Training	0.9
Opponent	DMARC	0.96
Opponent	Breached Passwords Audit	0.67
Opponent	MFA Rate Limiting	0.99
Opponent	Session Timeout	0.8
Opponent	Conditional Access/ Ip Reputation	0.99
Opponent	Endpoint Detection and Response	0.95
Opponent	AI-Driven Behavioral Analysis	0.82
Opponent	MFA and Account Lockout	0.99
Opponent	Presence Detection	0.74

Rock Paper Scissor Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#include <time.h>

/*

 * Plays one round of Rock-Paper-Scissors.

 * Updates the player's score via pointer.

 */

void play(int *score){

    char computer_selection;

    char dummy_flag[2] = {'c','l'};   // Dummy variable (for stack layout)

    char user_selection;              // Stores user's choice (R, P, or S)

    char choices[3] = {'R','P','S'};  // Possible choices
```

```
int won = 0;                    // Win flag (0 = lose, 1 = win)

/* Randomly select from R or P or S */
computer_selection = choices[rand() % 3];

printf("Choose (R)ock,(P)aper, or (S)cissor: ");
scanf(" %c", &user_selection);   // Read a single character safely, doesn't exploit buffer overflow
user_selection = toupper(user_selection);

/* Input validation */
if(user_selection != 'R' && user_selection != 'P' && user_selection != 'S'){
   printf("\t! Invalid input. Please choose R, P, or S.\n");
   return;
}

printf("You: %c - Computer: %c\n", user_selection, computer_selection);

/* Tie Condition*/
if(user_selection == computer_selection){
   printf("\t= It's a tie!\n");
   return;
}
```

```
    /* Winning conditions */
    if(user_selection == 'R' && computer_selection == 'S'){
        won = 1;
    }
    if(user_selection == 'P' && computer_selection == 'R'){
        won = 1;
    }
    if(user_selection == 'S' && computer_selection == 'P'){
        won = 1;
    }

    /* Score Result */
    if(won){
        (*score)++;
        printf("\t+ You won!\n");
    } else {
        printf("\t- You lost!\n");
    }
}

int main(){
    int played = 0;   // Total games payed
    char input = 'c'; // Menu input
```

```
    int win = 0;      // wins

    while(1){
        printf("Press P to play,E for scores, or X to exit: ");
        scanf(" %c", &input);
        input = toupper(input);

        if(input == 'X'){
            break;   // Exit program
        }
        else if(input =='P'){
            played++;
            play(&win);   // Play one round
        }
        else if(input =='E'){
            printf("\twon:%d\n\tplayed:%d\n", win, played);
        }
        else{
            printf("Invalid input!\n");
        }
    }
    return 0;
}
```